Đề xuất cải tiến hiệu năng

Service: Toàn hệ thống **Rà soát:** 2026-10-30 **Dự án:** Chande

Đây là tài liệu duy nhất gom mọi đề xuất cải tiến hiệu năng: kiến trúc, môi trường đo, chi phí, đối chiếu với ADR và lộ trình. Số liệu gốc nằm trong báo cáo kiểm thử tải; test case và ngưỡng nằm trong kế hoạch kiểm thử hiệu năng.

Các đề xuất bám theo kiến trúc mục tiêu và ADR, không thêm công nghệ mà tài liệu đó đã loại (Kafka, sharding, service mesh). Mọi con số "sau cải tiến" là ước tính, phải đo lại để xác nhận.

1. Tóm tắt

Hiện tại hệ thống đạt ngưỡng ở khoảng **1000 tài xế, 2 chuyến/giây**. Trần nằm ở cách các worker đọc hàng đợi, không phải ở phần cứng: ở 3 chuyến/giây, mọi pod vẫn dùng dưới 0,3 core. Vì vậy phần lớn cải tiến là sửa code nhỏ hoặc đổi cấu hình, gần như không tốn thêm tài nguyên.

Bước	Nội dung	Công sức (ước tính)	Thêm tài nguyên	Hiệu quả kỳ vọng
1	Gỡ ba hàng đợi theo nhịp (A1–A3), sửa đường đo (E1, E2)	khoảng 6–7 ngày người	Không	Trần từ khoảng 2,5 lên khoảng 6–10 chuyến/giây; 3000 chuyến gán xong trong khoảng 5–8 phút thay vì 15–20 phút
2	Rate limit theo IP thật, driver-api 2 replica, làm mới token có jitter (A4, B4, B5)	khoảng 3 ngày người	khoảng 0,1 GB RAM	Đăng nhập không còn dùng chung một quota; tài xế lên ca đồng loạt không làm driver-api bị restart
3	Các mục đã có trong lộ trình: P1.1, P1.3, P1.4, P2.5 (B1–B3, B6)	đã tính trong Phase 1–2	khoảng 1,2 GB RAM (observability và replica)	Chi phí mỗi lần tìm không còn tăng theo mật độ tài xế; có số liệu phía server để tìm trần tiếp theo
4 (tuỳ chọn)	Thuê cụm thử nghiệm trong 1 ngày (E4)	khoảng 2 ngày người	Ước tính 5–20 USD	Đo được 3000–5000 tài xế, điều mà máy local không làm được

Muốn gán 3000 chuyến trong khoảng 2 phút (25 chuyến/giây) thì cần cả bước 1–3 và nhiều replica trên nhiều máy. Mục tiêu đó vượt quá hạ tầng demo trên một MacBook (mục 8).

2. Hiện trạng đo được

Đo trên Docker Desktop Kubernetes một node, đi qua `kubectl port-forward`, ngày 2026-10-28 và 2026-10-29. Chi tiết trong báo cáo.

Chỉ số	Kết quả
Kịch bản cao nhất đạt ngưỡng	1000 tài xế, 2 chuyến/giây: tới `ASSIGNED` p95 4,0 giây, báo giá p95 22 ms, 0% lỗi

Chỉ số	Kết quả
2,5 chuyến/giây (1000 tài xế)	Mọi chuyến được gán nhưng p95 13,3 giây, báo giá p95 535 ms: vượt ngưỡng
3 chuyến/giây (1000 tài xế)	Hàng đợi dài dần (p95 30,6 giây) trong khi mọi pod dưới 0,3 core
2000 tài xế	Ở 2 chuyến/giây có 10% chuyến chờ tới 5,7 phút; ở 2,5 chuyến/giây PostgreSQL 0,62 core, driver-api 0,4 core
Kết nối tài xế	Gãy lần lượt ở edge (khoảng 500 WebSocket), edge OOM (128 MiB), driver-api khi lên ca đồng loạt, rồi môi trường đo (khoảng 4000 kết nối)
Thời gian tới `ASSIGNED` khi rảnh	khoảng 3 giây, do 3–4 vòng chờ nhịp 1 giây
3000 khách, 3500 tài xế	Cần khoảng 13–20 phút để gán hết; chưa đo trọn vì `cri-dockerd` sập

3. Ràng buộc chi phí

Loại chi phí	Ràng buộc (theo kiến trúc mục tiêu)
Công sức	3 người (1 Java, 2 Node/TS), mỗi phase 2–3 tuần, đã có lịch Phase 1 và Phase 2
Tài nguyên	Demo trên MacBook 16 GB, 10 core; VM Docker khoảng 7,7–8 GB; ngân sách sau Phase 2 khoảng 5,2 GB
Tiền	Không có ngân sách cloud thường xuyên; chỉ có thể thuê ngắn hạn cho một buổi đo
Công nghệ	Giữ PostgreSQL, Redis, RabbitMQ; chỉ thêm bộ observability

Thứ tự ưu tiên vì vậy là: sửa code nhỏ, rồi cấu hình, rồi tài nguyên. Không thêm thành phần hạ tầng mới ngoài observability đã có trong kế hoạch.

4. Cải tiến kiến trúc

4.1 Nhóm A: rẻ, làm ngay (code nhỏ hoặc cấu hình)

ID	Cải tiến	Giải quyết	Chỗ sửa	Công sức	Rủi ro	Hiệu quả ước tính
A1	Outbox Trip: lặp ngay khi lô đầy thay vì luôn nghỉ `OUTBOX_POLL_INTERVAL_MS`; tăng `OUTBOX_BATCH_SIZE` lên 100	Trần khoảng 2,7 chuyến/giây: 20 tin mỗi nhịp, 7,3 tin mỗi chuyến	`trip-service/src/bootstrap/worker-runtime.ts`, env	0,5–1 ngày	Thấp: claim đã dùng `SKIP LOCKED` và partial index (0,03 ms)	Outbox Trip không còn là trần
A2	Publisher Matching: gửi tới khi hết tin (có trần mỗi lượt, ví dụ 200), chạy theo vòng riêng, không chờ worker	Trần khoảng 2,8 chuyến/giây: 10 tin mỗi nhịp, viết cứng; offer chờ publish 1–6 giây	`matching-service/src/infrastructure/publisher.ts`, `src/worker.ts`	1 ngày	Thấp: vẫn một tin mỗi lần, có confirm	Đoạn offer → publish từ khoảng 1 giây xuống dưới 100 ms

ID	Cải tiến	Giải quyết	Chỗ sửa	Công sức	Rủi ro	Hiệu quả ước tính
A3	Worker Matching: nhận job mới ngay khi có chỗ trống, thay vì mỗi nhịp một lô rồi chờ job chậm nhất; giữ offer bằng hạn `expires_at` thay vì kiểm tra lại mỗi 1 giây; `WORKER_CONCURRENCY=16`	Trần kế tiếp sau A1–A2: khoảng 8 job/giây, mỗi chuyến cần 2–4 job	`matching-service/src/application/worker.ts`, `match.ts`	1,5–2 ngày	Trung bình: chạm logic lease; cần test offer hết hạn	Khoảng 2–3 lần thông lượng worker; bỏ phần lớn job kiểm tra lại
A4	Rate limit theo IP thật: nginx gửi `X-Forwarded-For` và `X-Real-IP` (hiện chưa gửi); Gateway bật `server.forward-headers-strategy` và chỉ tin edge	Mọi người dùng chung một quota theo IP của edge; mặc định chỉ 5 lần/phút	`deploy/nginx.conf`, Gateway `application.yaml`	0,5 ngày	Thấp; phải chặn client tự gửi header giả	Rate limit đúng theo từng người dùng
A5	Hạ nhịp poll khi rảnh: `WORKER_SCAN_MS=250`, `OUTBOX_POLL_INTERVAL_MS=250`	Khoảng 3 giây chờ cố định mỗi chuyến	env trong `deploy/stack.cjs`	0,5 ngày, gồm đo lại	Thêm khoảng 8 query/giây khi rảnh, không đáng kể	Thời gian tới `ASSIGNED` khi rảnh từ khoảng 3 xuống khoảng 1 giây
A6	Dọn bảng outbox đã gửi, giữ 24 giờ	`outbox_deliveries` tăng không giới hạn (110.464 dòng sau các lần đo)	Job dọn trong trip-worker và matching-worker	0,5 ngày	Thấp	Tránh suy giảm khi chạy lâu
A7	Ghi mã lỗi trong `matching_job_retry`, giới hạn backoff khi gặp `ROUTING_BUSY`	Lỗi bị nuốt bằng `catch {}`; 37 chuyến chờ tới 5,7 phút chưa rõ nguyên nhân	`matching-service/src/application/worker.ts`	0,5 ngày	Thấp	Tìm được nguyên nhân các chuyến chờ lâu
A8	Đặt rate limit Routing theo năng lực OSRM tự host trong cấu hình triển khai thật (local đã đặt 20000 element/phút, 50 request/giây, burst 20)	Mặc định 100 element/phút: baseline chỉ 1/270 chuyến được gán	Cấu hình triển khai, hoặc mặc định riêng cho chế độ OSRM tự host	0,5 ngày	Thấp; phải đo năng lực OSRM thật trước khi chốt	Không còn `ROUTING_BUSY` ở tải bình thường
A9	Đưa cấu hình edge đã nâng trên local vào cấu hình triển khai thật: `worker_processes auto`, `worker_connections` và RAM tính theo số tài xế online lúc cao điểm (khoảng 2 kết nối và vài chục KB mỗi WebSocket)	Edge gãy ở khoảng 500 WebSocket và OOM ở 128 MiB	`deploy/nginx.conf`, `deploy/generate.cjs`, cấu hình thật	0,5 ngày	Thấp	Edge không còn là điểm gãy đầu tiên khi tăng tài xế

Tổng nhóm A khoảng 6–7 ngày người, chia được cho 2 người Node và 1 người Java trong một tuần. Không cần thêm RAM, ngoài RAM của edge tính theo số kết nối.

4.2 Nhóm B: trung bình, phần lớn đã có trong lộ trình

ID	Cải tiến	Giải quyết	Liên quan lộ trình	Công sức	Thêm tài nguyên	Hiệu quả ước tính
B1	Matching tự lấy tối đa 200 vị trí, kiểm eligibility theo lô, chỉ tính ETA cho top 25	Realtime kiểm eligibility cho mọi tài xế trong 2 km (khoảng 220 ở 2000 tài xế); giới hạn 50 phải khớp ở 3 service	P1.1, ADR-0001	Đã tính trong Phase 1	Không	Chi phí mỗi lần tìm bị chặn trên; matrix 25 thay vì 50 element

ID	Cải tiến	Giải quyết	Liên quan lộ trình	Công sức	Thêm tài nguyên	Hiệu quả ước tính
B2	Gộp đích `gateway` và `notification` của outbox Trip thành một đích `bus` trên RabbitMQ	7,3 tin mỗi chuyến, trong đó mỗi lần đổi trạng thái tốn 2 tin	P1.3, ADR-0002	Đã tính trong Phase 1	Không	Còn khoảng 4 tin mỗi chuyến
B3	2 replica matching-worker	Một worker là điểm nghẽn và điểm lỗi duy nhất	P2.5 (lease đã có sẵn)	Đã tính trong Phase 2	khoảng 0,08 GB	Gần gấp đôi thông lượng worker nếu PostgreSQL chưa nghẽn
B4	2 replica driver-api; Gateway chuyển `X-Driver-Client-IP` và `X-Driver-Proxy-Token` cho Driver	driver-api bão hòa 1 core khi lên ca đồng loạt; event loop nghẽn quá 30 giây thì liveness restart (probe không chạm DB); OTP chỉ 10 lần/phút cho cả hệ thống	Mới, bổ sung cho P2.5	1–1,5 ngày (Java + deploy)	khoảng 0,1 GB	Lên ca đồng loạt không làm sập driver-api; OTP giới hạn đúng từng IP
B5	App tài xế làm mới token trước khi hết hạn và kết nối lại có jitter	Realtime ngắt socket đúng lúc JWT hết hạn, nên tài xế đăng nhập cùng lúc sẽ cùng kết nối lại mỗi 15 phút	Mới (app)	1 ngày	Không	Tránh đợt kết nối lại đồng loạt
B6	Observability: metrics HTTP, độ dài và tuổi tin outbox, event loop lag, HikariCP, hàng đợi job Matching; dashboard	Chưa có số liệu phía server; các đợt chậm p99 2–3 giây chưa rõ nguyên nhân	P1.4, ADR-0003	Đã tính trong Phase 1	khoảng 0,8 GB	Là điều kiện để đo các bước sau

4.3 Nhóm C: đắt hoặc để sau

ID	Cải tiến	Khi nào cần	Chi phí
C1	Nhiều replica cho trip-api, gateway-api, realtime-api (Socket.IO Redis adapter)	Khi nhóm A–B đã xong mà một process Node hoặc JVM chạm khoảng 1 core	Mỗi replica Node khoảng 0,1 GB, mỗi replica JVM khoảng 0,5 GB; Realtime nhiều replica nằm trong P2.5
C2	PgBouncer, tách PostgreSQL theo service hoặc thêm replica đọc	Khi PostgreSQL là pod nóng nhất ở tải mục tiêu (đã thấy 0,6–0,9 core ở 2000 tài xế)	Thêm thành phần vận hành; đo `pg_stat_statements` trước
C3	Realtime lưu sẵn loại xe và trạng thái tài xế trong Redis để lọc tại chỗ (phương án ADR-0001 đã để sau)	Khi mật độ tài xế cao làm việc lọc ở Matching và Driver thành nghẽn	Cần event từ Driver sang Realtime
C4	Realtime đẩy offer kèm dữ liệu, bỏ lời gọi `GET /internal/matching/offers/:id` cho mỗi offer	Hiện đoạn publish → accept chỉ 6–8 ms, chưa cần	Đổi contract event offer
C5	Cụm production nhiều node, autoscaling	Khi có tải thật	Ngoài phạm vi đồ án

5. Cải tiến môi trường đo

ID	Cải tiến	Giải quyết	Công sức	Chi phí
E1	Đưa tải vào không qua `kubectl port-forward`. Rẻ nhất: chạy stack bằng Docker Compose (`node deploy/run.cjs docker-up`, edge đã bind `127.0.0.1:18080`) và cho `accounts.cjs` hỗ trợ `docker exec`. Hoặc chạy bộ giả lập và k6 dưới dạng Job trong cụm	Port-forward làm control plane dùng 2–5 core từ khoảng 4000 kết nối; `cri-dockerd` sập ở khoảng 3500 kết nối	1–2 ngày	0 đồng
E2	Mặc định GPS 10 giây như app; thêm kịch bản k6 nhận sự kiện qua `/ws` thay cho poll	Bộ đo nặng hơn thực tế: GPS gấp 3, poll mỗi 300 ms khuếch đại tải của Gateway, Trip, PostgreSQL khi chờ lâu	1 ngày	0 đồng
E3	Bật `pg_stat_statements` trên PostgreSQL local	Chưa biết truy vấn nào nóng khi PostgreSQL lên 0,6–0,9 core	0,5 ngày	Không đáng kể
E4	Thuê cụm thử nghiệm trong 1 ngày: 3 node ứng dụng khoảng 4 vCPU / 16 GB, 1 node PostgreSQL, 2 máy tạo tải, dựng bằng manifest trong `deploy/`	Máy local không đo được 3000–5000 tài xế và 3000 chuyến cùng lúc	khoảng 2 ngày người (dựng, đo, gỡ)	Ước tính 6 máy × 8 giờ × 0,1–0,4 USD/giờ, tức khoảng 5–20 USD, cộng phí Kubernetes quản lý nếu có. **Cần kiểm tra bảng giá hiện tại**
E5	Cấp VM Docker ít nhất 10 GB khi chạy observability cùng bài đo	Ngân sách sau Phase 2 khoảng 5,2 GB; bài đo 2000 tài xế dùng thêm khoảng 2–3 GB	0	Giảm RAM còn lại cho macOS và emulator

E1 và E2 nên làm trước khi đo lại nhóm A, nếu không số đo sau tối ưu vẫn bị môi trường đo giới hạn.

6. Đối chiếu với ADR

Các ADR 0001–0007 đều đang ở trạng thái **Đề xuất**, nên có thể bổ sung trước khi chấp nhận. Bảng dưới cho biết ADR nào dùng được cho hiệu năng, và còn thiếu gì so với kết quả đo.

ADR	Dùng được cho	Còn thiếu so với kết quả đo	Cần đo khi triển khai
0001 **Matching tự tìm ứng viên**	**B1, trực tiếp.** `NEARBY_LIMIT=200` chặn eligibility ở tối đa 2 lô 100; Driver không gọi occupancy; matrix còn 25 element; gỡ việc ba service cùng giữ giới hạn 50	Không nói tới rate limit Routing: với mặc định 100 element/phút, 25 element mỗi lần tìm vẫn chỉ được khoảng 4 lần/phút (A8). Không nói tới nhịp worker (A3)	Trần thông lượng ở 1000 và 2000 tài xế; CPU PostgreSQL và driver-api mỗi lần tìm

ADR	Dùng được cho	Còn thiếu so với kết quả đo	Cần đo khi triển khai
0002 **Trục sự kiện RabbitMQ**	**B2, trực tiếp.** Tin outbox Trip mỗi chuyến từ khoảng 7,3 xuống khoảng 4,3; event không mất khi Gateway tắt	Worker outbox vẫn 20 tin mỗi nhịp 1 giây, nên trần chỉ lên khoảng 4–5 chuyến/giây nếu không có A1. Publisher confirm làm mỗi lần gửi chậm hơn POST HTTP nội bộ. Publisher offer của Matching (A2) nằm ngoài phạm vi	Độ dài và tuổi tin outbox Trip; thời gian publish và confirm
0003 **Observability**	**B6, E3.** Dashboard bắt buộc đã gồm funnel matching và độ trễ outbox, đúng hai chỗ báo cáo phải tự tính từ DB	Chưa có exporter PostgreSQL (`pg_stat_statements`), Redis, RabbitMQ, nginx; chưa có event loop lag, HikariCP, hàng đợi job Matching; chưa đưa số liệu k6 vào cùng Grafana	RAM thật của bộ observability (ADR ước tính khoảng 0,8 GB)
0004 **Giới hạn thời gian tìm xe**	**Bảo vệ khi quá tải:** chuyến không treo vô hạn và không tiếp tục chiếm job worker	Chưa tính hệ quả với tải đột biến: với trần hiện tại và hạn 3 phút, trong kịch bản 3000 khách chỉ khoảng 450 chuyến được gán, còn khoảng 2550 chuyến bị huỷ với `NO_DRIVER_FOUND` (mục 8)	Số chuyến `NO_DRIVER_FOUND` trong kịch bản đợt; tranh chấp accept ở giây cuối dưới tải
0005 **Payment**	Không gỡ nghẽn; thêm lời gọi đồng bộ Trip → Payment khi tạo chuyến bằng ví và một consumer `trip.events`	Chưa có ngưỡng thời gian cho bước Try; Payment chậm thì `POST /trips` chậm theo	p95 `POST /trips` có và không có ví; hai chuyến đồng thời không giữ vượt số dư
0006 **Giá động H3**	Không gỡ nghẽn; báo giá thêm một lần đọc Redis, cung tính trước mỗi 60 giây	Bộ đếm cầu tăng theo mỗi `trip.searching`, nên cũng tăng theo tải đột biến	p95 báo giá trước và sau surge; CPU Price
0007 **Admin web**	Không ảnh hưởng luồng nóng; thống kê lấy từ Grafana	Không	Không bắt buộc

ADR mới. Chưa ADR nào trong 0001–0007 xử lý trần chính, tức cách outbox và worker đọc hàng đợi theo nhịp. ADR-0008 (Đề xuất) phủ A1, A2, A3, A5:

- outbox Trip lặp khi lô đầy;
- publisher Matching chạy vòng riêng tới khi hết tin;
- worker Matching chạy theo pool liên tục, giữ offer bằng `expires_at`;
- nhịp rảnh 250 ms.

ADR-0008 nên làm cùng ADR-0002, vì cả hai sửa cùng worker outbox của Trip.

7. Lộ trình đề xuất

Thời điểm	Việc	Đo lại bằng	Mục tiêu (ước tính, cần xác nhận)
Ngay, 1 tuần	Duyệt ADR-0008; E1, E2; A1, A2, A5, A6, A7, A8	Trần thông lượng: 1000 tài xế, 2 → 4 → 8 chuyến/giây	Đạt ngưỡng ở khoảng 4–5 chuyến/giây; khi rảnh, tới `ASSIGNED` khoảng 1 giây
Ngay sau đó	A3, A4, A9	Như trên, cộng kịch bản đăng nhập và tăng quy mô tài xế	Đạt ngưỡng ở khoảng 6–10 chuyến/giây; không còn 429 cho người dùng hợp lệ

Thời điểm	Việc	Đo lại bằng	Mục tiêu (ước tính, cần xác nhận)
Phase 1	B1, B2, B6 (P1.1, P1.3, P1.4)	Trần thông lượng với 2000 tài xế; dashboard	Trần không giảm khi tăng từ 1000 lên 2000 tài xế
Phase 2	B3, B4, B5 (cùng P2.5)	Tài xế lên ca đồng loạt; chạy lâu 2 giờ	Không restart; RAM ổn định
Trước bảo vệ (tuỳ chọn)	E4	Kịch bản 3500 tài xế + 3000 chuyến	Ghi lại thời gian gán hết 3000 chuyến trên cụm thật

Với mỗi cải tiến, chạy lại đúng kịch bản trước đó và so trong một bảng trước/sau. Mỗi lần đo chỉ đổi một thay đổi (kế hoạch, giai đoạn 9).

8. Kỳ vọng cho kịch bản 3000 khách

Trạng thái	Thông lượng ước tính	Gán hết 3000 chuyến	Được gán trong 3 phút nếu có ADR-0004
Hiện tại	khoảng 2–2,5 chuyến/giây	15–20 phút	khoảng 450
Sau nhóm A	khoảng 6–10 chuyến/giây	5–8 phút	khoảng 1000–1800
Sau nhóm A + B, vẫn trên một MacBook	khoảng 10–15 chuyến/giây; khi đó PostgreSQL và các process một luồng có thể thành giới hạn	3–5 phút	khoảng 1800–2700
3000 chuyến trong khoảng 2 phút	cần khoảng 25 chuyến/giây	Cần thêm nhóm C và nhiều máy; ngoài phạm vi demo	3000

Các con số suy ra từ trần đo được, các giới hạn trong code (lô outbox, nhịp worker) và CPU đo được; chưa được đo sau khi sửa. Khi có hạn tìm xe 3 phút (ADR-0004), chuyến vượt trần bị huỷ với `NO_DRIVER_FOUND` thay vì chờ tới lượt. Số chuyến được gán xấp xỉ bằng thông lượng × 180 giây, nên nâng thông lượng quyết định trực tiếp số khách được phục vụ trong một đợt đột biến.

9. Nối điểm nghẽn với cải tiến

Điểm nghẽn trong báo cáo	Cải tiến
1. Hai hàng đợi outbox giới hạn khoảng 3 chuyến/giây	A1, A2, A3, A5 (ADR-0008), B2, B3
2. Rate limit đăng nhập tính theo IP của proxy	A4, B4
3. driver-api quá tải khi nhiều tài xế lên mạng cùng lúc	B4, B5
4. Edge nginx chưa cấu hình cho WebSocket số lượng lớn	A9
5. Realtime kiểm eligibility mọi tài xế trong 2 km, giới hạn 50 phải khớp ở ba service	B1, C3
6. Rate limit Routing mặc định	A8

Điểm nghẽn trong báo cáo	Cải tiến
7. Nhịp poll 1 giây của worker Matching	A3, A5
8. Lỗi matching bị nuốt và backoff tới 30 giây	A7
9. Poll trạng thái làm khuếch đại tải khi chờ lâu	E2
10. Thêm một bước HTTP cho mỗi offer	C4
Môi trường đo: port-forward, `cri-dockerd`	E1, E4, E5
Chưa có số liệu phía server	B6, E3

10. Không đề xuất

Không làm	Lý do
Kafka, thay RabbitMQ	Trần hiện tại là cách worker đọc hàng đợi, không phải message broker; kiến trúc mục tiêu đã loại vì tốn RAM
Tăng CPU/RAM cho pod	Ở 3 chuyến/giây mọi pod dưới 0,3 core; thêm tài nguyên không nâng trần
Batch matching, geo-sharding, service mesh	Đã nằm trong mục không làm của kiến trúc mục tiêu; chưa phải nghẽn
Cache báo giá	Báo giá chỉ chậm khi hệ thống đã quá tải; nguyên nhân gốc là hàng đợi và poll

11. Giả định và việc cần xác minh

1. Ước tính công sức tính cho người đã quen code của service đó, chưa gồm thời gian review.
2. Hiệu quả của A1–A3 phụ thuộc độ trễ HTTP thật giữa các service ở tải cao, hiện chưa đo được vì thiếu metrics (B6).
3. Giá cloud ở E4 là khoảng tham khảo; cần kiểm tra bảng giá hiện tại và hạn mức tài khoản sinh viên nếu có.
4. Năng lực OSRM thật (A8) và số tài xế online lúc cao điểm (A9) cần đo hoặc chốt trước khi đặt cấu hình thật.
5. Tải mục tiêu (số tài xế, số chuyến giờ cao điểm) chưa được chốt (kế hoạch, mục 10). Nếu mục tiêu chỉ là demo trước hội đồng, nhóm A cùng các mục đã có trong lộ trình là đủ.